



React Suspense

+ Cache System

Series 25 of React Concepts

1. What is Suspense?

★ Concept

Suspense lets React **pause rendering** until data is ready, instead of showing broken UI.

```
<Suspense fallback={<Spinner />}>
  <UserProfile />
</Suspense>
```

🎯 Interview Questions

Conceptual

- What problem does Suspense solve?
- How is Suspense different from loading states?

Tricky

- Does Suspense fetch data?
- What happens when data is not ready?

Coding

- Implement a Suspense boundary

2. Why Suspense is Needed

❌ Before Suspense

We manually do:

```
if (!data) return "Loading..."
```

- ❌ Scattered logic
- ❌ Nested conditions
- ❌ Hard to coordinate

✓ After Suspense

React pauses rendering instead.

```
<Suspense>...</Suspense>
```

- ✓ Clean components
- ✓ Centralized loading
- ✓ Easy coordination

🎯 Questions

Conceptual

- Why is manual loading handling bad?
- What is waterfall fetching problem?

Tricky

- Does Suspense replace useEffect?
- Is Suspense only for data?

Coding

- Create nested suspense boundaries

3. Suspense is NOT Fetching

✗ Misconception

Suspense does NOT fetch data.

It waits for something that **throws a promise**.

```
// Component throws promise
function User() {
  const user = fetchUser(); // throws promise
  return <div>{user.name}</div>;
}

// Suspense catches it
<Suspense fallback={<Loading />}>
  <User />
</Suspense>
```

Questions

Conceptual

- How does Suspense detect pending data?
- Why does it throw a promise?

Tricky

- What happens if promise rejects?
- Can you use Suspense without async?

Coding

- Create a wrapper that throws promise

4. React Cache System

React Cache API

cache() creates a cached function.

```
import { cache } from "react";

const getUser = cache(async (id) => {
  return fetch(`/api/user/${id}`)
    .then(r => r.json());
});

// Same ID = cached result
await getUser(1); // fetches
await getUser(1); // cached ✨
```

Questions

Conceptual

- What is cache() and why is it useful?
- How does caching reduce waterfalls?

Tricky

- Does cache persist across requests?

Coding

- Create a cached fetch function

5. Fetching Inside Components

🌟 Server Components Example

Async components fetch data directly!

```
async function User() {  
  const user = await getUser();  
  return <div>{user.name}</div>;  
}  
  
// Usage  
<Suspense fallback={<Loading />}>  
  <User />  
</Suspense>
```

🎯 Questions

Conceptual

- Why can Server Components fetch data directly?
- Why no useEffect?

Tricky

- Is Server Component blocking?

Coding

- Build a server component that fetches posts

6. Suspense + Fetch Integration

💡 Pattern

```
async function Posts() {
  const posts = await getPosts();
  return posts.map(p =>
    <Post key={p.id} data={p} />
  );
}

<Suspense fallback={<PostsSkeleton />}>
  <Posts />
</Suspense>
```

🎯 Questions

Conceptual

- How does Suspense integrate with Server Components?
- What is streaming?

Tricky

- Difference: server fetch vs client fetch?

Coding

- Wrap Posts with Suspense

7. Waterfall vs Parallel

❌ Waterfall (BAD)

Component waits for another → slow.

```
A finishes
↓
B starts
↓
C starts
```

🕒 Total: A + B + C time

✅ Parallel (GOOD)

Fetch all at once.

```
A, B, C
all start together
```

⚡ Total: max(A, B, C) time

🎯 Questions

Conceptual

- What is waterfall fetching?
- How does Suspense fix it?

Tricky

- How does caching make parallel possible?

Coding

- Convert waterfall to parallel

8. Boundary Placement



Tip

Place boundary **closest to where delay might happen**.

```
// Good: Granular control
<Layout>
  <Suspense fallback={<NavSkeleton />}>
    <Nav />
  </Suspense>

  <Suspense fallback={<ContentSkeleton />}>
    <Content />
  </Suspense>
</Layout>
```



Questions

Conceptual

- Why multiple Suspense boundaries?
- What are fallback strategies?

Tricky

- Static fallback vs skeleton loader?

Coding

- Implement nested fallbacks

9. Error Boundaries

! Pair Together

Always combine Suspense with ErrorBoundary

```
<ErrorBoundary fallback={<Error />}>  
<Suspense fallback={<Spinner />}>  
<User />  
</Suspense>  
</ErrorBoundary>
```

Questions

Conceptual

- Why are error boundaries needed?
- What errors do they catch?

Tricky

- Do they catch async errors?

Coding

- Build custom ErrorBoundary

10. Streaming UI

Streaming

HTML is sent chunk by chunk.

User sees UI as soon as possible.

```
// Server sends:  
1. Layout HTML immediately  
2. Nav when ready  
3. Content when ready  
  
// User sees progressive loading!
```

Questions

Conceptual

- What is streaming in React?
- How does Suspense enable streaming?

Tricky

- Can streaming happen without Suspense?

Coding

- Streaming SSR example (conceptual)

11. Suspense + Lazy

Code-splitting

```
const Chart = React.lazy(  
  () => import("./Chart")  
);  
  
<Suspense fallback={<ChartLoader />}>  
  <Chart data={data} />  
</Suspense>
```

Questions

Conceptual

- Why combine lazy with Suspense?
- What is bundle splitting?

Tricky

- Does lazy load on server?

Coding

- Lazy load a component

12. Data Fetching Libraries

Popular approaches:

React Query

SWR

Relay

Urql

```
// React Query with Suspense
const { data } = useQuery({
  queryKey: ['user'],
  queryFn: fetchUser,
  suspense: true
});
```

Questions

Conceptual

- Why do we still use React Query after Suspense?
- What does React Query add?

Tricky

- When NOT to use Suspense?

Coding

- Suspense with SWR example

13. Common Mistakes

✗ Bad

```
useEffect(() => {  
  fetchData()  
}, [])
```

Waterfall, race conditions, manual loading states

✓ Good

```
async function Page() {  
  const data = await fetch()  
  return <UI />  
}
```

Parallel, clean, automatic loading

Questions

Conceptual

- What are pitfalls of useEffect fetching?
- Why is Server fetching faster?

Tricky

- What does hydration have to do with it?

Coding

- Fix an anti-pattern

14. Interview Traps

⚠ Common trick questions:

? Does Suspense fetch data?

👉 No. It just waits for thrown promises.

? Does Suspense replace React Query?

👉 No. They complement each other.

? Does Suspense work on client components?

👉 Only if data fetch throws a promise.

? Can Suspense handle errors?

👉 No. Use ErrorBoundary for that.

🎯 Questions

Advanced

- Explain throwing promise pattern
- Can Suspense work with file uploads?

15. Final Takeaways

Key Points

- ✓ Suspense = wait mechanism, not data fetcher
- ✓ Cache = avoids waterfalls & duplicate fetches
- ✓ Server Components fetch data directly
- ✓ Streaming = faster UX, progressive loading
- ✓ Place boundaries carefully for best UX
- ✓ Always pair with ErrorBoundary

Short Interview Pitch:

"Suspense solves loading state chaos. It pauses rendering until data is ready, removes waterfalls, and enables streaming. I combine it with cache() and Server Components for fast, clean data flows."

Pro Tip

When interviewer asks "How do you handle data fetching?" → mention Suspense, cache(), Server Components, and streaming. This shows you're up-to-date with modern React patterns.

Master React Suspense!



LIKE



SHARE



FOLLOW

[@mycodeshala](#)